

Video Metadata Template

Container Security Course - Video Description Standard

This template provides the structure for all video metadata in the course. Each video should have complete, searchable, and informative metadata to enhance discoverability and student experience.

Template Structure

Video Title Format

Module [NUMBER]: [DESCRIPTIVE TITLE] | Container Security Course

Examples:

- Module 3.1: Introduction to Seccomp | Container Security Course
 - Module 3.2: Creating Custom Seccomp Profiles | Container Security Course
 - Module 3.3: Advanced Seccomp Troubleshooting | Container Security Course
-

Complete Metadata Template

```

# =====
# VIDEO METADATA
# =====

# Basic Information
video_id: "M[module]V[video]"           # Example: M3V1, M3V2, M3V3
module_number: [number]                  # Example: 3
video_number: [number]                   # Example: 1
title: "[Full Video Title]"              # See title format above
short_title: "[Concise Title]"            # Example: "Intro to Seccomp"

# Duration & Technical Specs
duration_minutes: [number]               # Example: 13
duration_formatted: "[MM:SS]"             # Example: "13:24"
resolution: "1920x1080"                  # Always Full HD
frame_rate: "30"                         # 30 FPS constant
file_format: "MP4"                       # Standard format

# Content Classification
difficulty_level: "[Easy/Medium/Hard]"    # Based on module progression
content_type: "[Theory/Demo/Lab/Mixed]"    # Primary content focus
screen_mix:                                # Percentage breakdown
  powerpoint: "[0-100]%"                  # Slides and diagrams
  terminal: "[0-100]%"                    # CLI and command execution
  code_editor: "[0-100]%"                 # VS Code, file editing
  browser: "[0-100]%"                     # Documentation, web tools

# Description (200-300 words)
description: |
  [2-3 sentence hook explaining what problem this video solves]

  [1 paragraph overview of what students will learn]

  [Bullet points of key topics covered]

  [Call-to-action: what students should do after watching]

# Learning Objectives (3-5 specific, measurable objectives)
learning_objectives:
  - "Explain [concept] and its role in [context]"
  - "Demonstrate [skill] using [tool/technique]"
  - "Analyze [scenario] to identify [issue]"
  - "Create [artifact] following [best practices]"
  - "Troubleshoot [problem] using [methodology]"

# Prerequisites
prerequisites:
  knowledge:
    - "[Previous module or concept]"
    - "[Required background knowledge]"
  technical:
    - "[Software/tools needed]"
    - "[Environment setup requirements]"
  optional:
    - "[Helpful but not required background]"

# Keywords (for searchability)
keywords:
  primary: ["[main topic]", "[key technology]", "[core concept]"
  secondary: ["[related tool]", "[related technique]", "[use case]"
  seo: ["[common search term]", "[alternative phrasing]"

```

```

# Hands-On Components
labs:
  - lab_id: "[Lab number]"
    title: "[Lab title]"
    duration: "[minutes]"
    difficulty: "[Easy/Medium/Hard]"
    validation: "[automated/manual/both]"

# Resources & Files
resources:
  slides: "[filename.pptx]"
  code_samples:
    - "[filename]"
  documentation:
    - title: "[Doc title]"
      url: "[URL]"
  tools_demonstrated:
    - name: "[Tool name]"
      version: "[version]"
      download_url: "[URL]"

# Timestamps (major sections for video chapters)
timestamps:
  - time: "00:00"
    title: "[Section title]"
    description: "[Brief description]"
  - time: "[MM:SS]"
    title: "[Section title]"
    description: "[Brief description]"

# Assessment
quiz_questions: [number]           # Number of MCQs for this video
passing_score: [percentage]        # Example: 75

# Versioning & Updates
version: "1.0"
date_created: "YYYY-MM-DD"
last_updated: "YYYY-MM-DD"
changelog:
  - version: "1.0"
    date: "YYYY-MM-DD"
    changes: "Initial release"

# Accessibility
captions: true                     # Always true
transcript_available: true         # Always true
audio_description: false          # Optional
sign_language: false              # Optional

# Notes for Instructors
instructor_notes: |
  [Any special considerations for delivering this content]
  [Common student questions or confusion points]
  [Suggested discussion topics]
  [Alternative examples or variations]

# Related Content
previous_video: "[Module X.Y: Title]"
next_video: "[Module X.Y: Title]"
related_videos:
  - "[Module X.Y: Related topic]"

```



Example: Completed Metadata (Module 3, Video 1)

```
# =====
# VIDEO METADATA - Module 3, Video 1
# =====

# Basic Information
video_id: "M3V1"
module_number: 3
video_number: 1
title: "Module 3.1: Introduction to Seccomp | Container Security Course"
short_title: "Intro to Seccomp"

# Duration & Technical Specs
duration_minutes: 13
duration_formatted: "13:24"
resolution: "1920x1080"
frame_rate: "30"
file_format: "MP4"

# Content Classification
difficulty_level: "Medium"
content_type: "Mixed"
screen_mix:
  powerpoint: "25%"
  terminal: "45%"
  code_editor: "25%"
  browser: "5%"

# Description
description: |
  Containers share the host kernel, which means a compromised container could
  potentially exploit kernel vulnerabilities. What if you could prevent containers
  from even making dangerous system calls? That's exactly what Seccomp does.

  In this video, you'll learn how Seccomp (Secure Computing Mode) filters system
  calls at the kernel level, dramatically reducing attack surface. We'll explore
  Docker's default Seccomp profile, see live demonstrations of blocked vs. allowed
  syscalls, and examine how Seccomp enforcement works in practice.

  Topics covered:
  • How Seccomp filters syscalls between user space and kernel
  • The difference between containers with and without Seccomp
  • Analyzing Docker's default Seccomp profile (JSON structure)
  • Viewing active Seccomp profiles using /proc filesystem
  • Understanding syscall blocking actions (ALLOW, ERRNO, KILL, LOG)
  • Real-world examples of dangerous syscalls blocked by default

  After this video, complete Lab 3.1 to experiment hands-on with Seccomp
  restrictions and see how blocked syscalls affect container behavior.

# Learning Objectives
learning_objectives:
  - "Explain how Seccomp filters system calls at the kernel level"
  - "Analyze and interpret the default Docker Seccomp profile structure"
  - "Demonstrate the difference between containers with and without Seccomp"
  - "Identify which syscalls are blocked by Docker's default profile and why"
  - "Use /proc filesystem to verify Seccomp enforcement status"

# Prerequisites
prerequisites:
  knowledge:
    - "Module 1: Container Security Fundamentals (privileged containers, capabilities)"
```

```

- "Module 2: Container Attack Vectors (kernel exploits, container escapes)"
- "Basic understanding of Linux system calls"
technical:
- "Docker 20.10+ installed"
- "Ubuntu 20.04+ or similar Linux distribution"
- "Terminal access with root/sudo privileges"
optional:
- "Familiarity with JSON syntax"
- "Basic kernel architecture knowledge"

# Keywords
keywords:
  primary: ["seccomp", "system call filtering", "container security", "kernel security"]
  secondary: ["docker security", "syscall", "linux security", "sandbox", "attack surface reduction"]
  seo: ["what is seccomp", "docker seccomp profile", "container syscall filtering", "seccomp tutorial", "how seccomp works"]

# Hands-On Components
labs:
- lab_id: "3.1"
  title: "Exploring Default Seccomp Behavior"
  duration: "15"
  difficulty: "Easy"
  validation: "automated"

# Resources & Files
resources:
  slides: "slides_module3_video1.pptx"
  code_samples:
    - "default-seccomp-profile.json"
  documentation:
    - title: "Docker Seccomp Documentation"
      url: "https://docs.docker.com/engine/security/seccomp/"
    - title: "Linux Seccomp Manual"
      url: "https://man7.org/linux/man-pages/man2/seccomp.2.html"
    - title: "Docker Default Seccomp Profile (GitHub)"
      url: "https://github.com/moby/moby/blob/master/profiles/seccomp/default.json"
  tools_demonstrated:
    - name: "Docker"
      version: "24.0+"
      download_url: "https://docs.docker.com/engine/install/"
    - name: "unshare"
      version: "util-linux 2.36+"
      download_url: "Built-in to most Linux distributions"

# Timestamps
timestamps:
- time: "00:00"
  title: "Introduction & Module Overview"
  description: "Course context and what you'll learn in this module"
- time: "01:30"
  title: "What is Seccomp?"
  description: "Syscall filtering fundamentals and kernel architecture"
- time: "04:00"
  title: "Live Demo: With vs Without Seccomp"
  description: "Comparing unconfined containers to default Seccomp"
- time: "06:30"
  title: "Docker Default Profile Deep Dive"
  description: "Analyzing the JSON structure and blocked syscalls"
- time: "09:30"
  title: "Viewing Active Profiles"

```



```

    description: "Using /proc filesystem to verify Seccomp status"
  - time: "11:30"
    title: "Lab 3.1 Introduction"
    description: "Hands-on exercise overview"
  - time: "12:30"
    title: "Wrap-Up & Key Takeaways"
    description: "Summary and preview of next video"

# Assessment
quiz_questions: 3
passing_score: 66

# Versioning & Updates
version: "1.0"
date_created: "2026-07-01"
last_updated: "2026-07-01"
changelog:
  - version: "1.0"
    date: "2026-07-01"
    changes: "Initial release"

# Accessibility
captions: true
transcript_available: true
audio_description: false
sign_language: false

# Notes for Instructors
instructor_notes: |
  Students often confuse Seccomp with capabilities or AppArmor. Emphasize that
  Seccomp operates specifically at the syscall boundary, while capabilities control
  privileged operations and AppArmor controls file/network access.

  Common questions:
  - "Why doesn't Docker block ALL syscalls except a whitelist?" → Explain backward
    compatibility concerns and the need to support diverse applications.
  - "Can Seccomp be bypassed?" → No, it's enforced by the kernel; but emphasize
    defense in depth.

  The unshare demonstration is particularly impactful—practice timing the comparison
  between unconfined and default profiles for maximum effect.

  If time permits, mention that Kubernetes uses the RuntimeDefault profile which
  maps to Docker/containerd defaults.

# Related Content
previous_video: "Module 2.3: Insecure Configurations and DoS Attacks"
next_video: "Module 3.2: Creating Custom Seccomp Profiles"
related_videos:
  - "Module 2.1: Container Escape Techniques"
  - "Module 4.1: AppArmor Fundamentals"

```

Metadata Usage Guidelines

For Video Platforms (YouTube, Vimeo, etc.):

- **Title:** Use the full title format
- **Description:** Use the description field, convert YAML to readable paragraphs

- **Tags:** Use all keywords (primary + secondary + SEO)
- **Chapters:** Use timestamps to create YouTube chapters
- **Playlist:** Group by module

For Learning Management Systems (LMS):

- **Title:** Use short_title for menu items
- **Duration:** Display duration_formatted
- **Prerequisites:** Show as expandable section before video
- **Learning Objectives:** Display prominently at the top
- **Resources:** Provide download links
- **Quiz:** Integrate quiz_questions count and passing_score

For Course Marketing:

- **Difficulty Level:** Help students self-assess readiness
- **Content Type:** Set expectations (theory vs. hands-on)
- **Screen Mix:** Demonstrate practical nature of content
- **Keywords (SEO):** Use for course landing page optimization

For Accessibility:

- Always include captions and transcripts
- Ensure instructor_notes mention any visual-only content that needs verbal description
- Consider providing transcript PDFs for offline study

Metadata Maintenance

When to Update:

- **Minor version (1.0 → 1.1):** Typo fixes, small clarifications, updated URLs
- **Major version (1.0 → 2.0):** Significant content changes, new demos, restructuring

Update Checklist:

- ☐ Update last_updated date
- ☐ Add entry to changelog
- ☐ Increment version appropriately
- ☐ Update related_videos if dependencies change
- ☐ Review prerequisites for accuracy
- ☐ Verify all resources URLs are still valid

Consistency Requirements:

- All videos in a module should use consistent keyword themes
- Duration estimates should be accurate within ± 1 minute
- Learning objectives should align with quiz questions
- Prerequisites should form a logical progression

Template Files Provided

For instructor convenience, the following template files are included:

1. **metadata_template_blank.yaml** - Empty template ready to fill
2. **metadata_example_complete.yaml** - Full example (Module 3 Video 1)
3. **metadata_validation_schema.json** - JSON schema for automated validation
4. **generate_metadata.py** - Python script to assist in bulk metadata generation

Using the Python Generator:

```
python3 generate_metadata.py \
  --module 3 \
  --video 2 \
  --title "Creating Custom Seccomp Profiles" \
  --duration 15 \
  --output metadata_m3v2.yaml
```

This generates a pre-filled template that you can then customize with specific details.

Quality Checklist

Before finalizing video metadata, verify:

- ☐ All required fields are populated
 - ☐ Description is 200-300 words and engaging
 - ☐ Learning objectives are specific and measurable (use action verbs)
 - ☐ Keywords include both technical terms and search phrases
 - ☐ Timestamps cover all major sections
 - ☐ Prerequisites accurately reflect required knowledge
 - ☐ Resources have valid, working URLs
 - ☐ Screen mix percentages add up to ~100%
 - ☐ Related videos exist and links are correct
 - ☐ Instructor notes include anticipated student questions
-

Last Updated: 2026-07-01

Version: 1.0

Maintained By: Course Production Team